Algoritmos de Consulta y Almacenamiento Columnar

IIC2440 — Procesamiento de Datos Masivos

Clase 03 — 2026-1

Agenda

1. **A** — Cómo la BD ejecuta nuestra consulta
2. **B** — Row stores vs Column stores
3. **C** — Demo
4. **Actividad Práctica** — PostgreSQL vs DuckDB

Parte A

Cómo la BD ejecuta nuestra consulta

El pipeline de una consulta

```
SQL query
    │
    ▼
┌──────────┐     ┌──────────────┐     ┌──────────────────┐     ┌──────────────┐
│  Parser  │──▶│ Optimizer  │──▶│ Physical Plan │──▶│ Execution │
└──────────┘     └──────────────┘     └──────────────────┘     └──────────────┘
                        │
                Usa estadísticas
                sobre las tablas
```

El optimizador elige **qué algoritmo** usar para cada operación basándose en costos estimados.

Operadores físicos

El plan de ejecución se compone de **operadores**. Los principales:

Operador	Qué hace
Scan	Leer datos de una tabla (o índice)
Join	Combinar filas de dos tablas
Sort	Ordenar filas por una o más columnas
Aggregation	Calcular `SUM`, `AVG`, `COUNT`, `GROUP BY`
Filter	Aplicar condiciones `WHERE` / `HAVING`

Cada operador tiene **distintos algoritmos** que el optimizador elige según el costo estimado en páginas de I/O.

Tipos de Scan

Algoritmo	Estrategia
Sequential Scan	Lee todas las páginas en orden (I/O secuencial)
Index Scan	Navega un índice B-tree → salta a las páginas relevantes (I/O random)
Index-Only Scan	Lee solo el índice, sin tocar la tabla

Tipos de Join

Algoritmo	Idea central
Nested Loop	Doble `for`: por cada fila de R, recorrer S (o buscar en índice)
Hash Join	Construir hash table con la tabla chica, recorrer la grande buscando matches
Sort-Merge Join	Ordenar ambas tablas por la llave y recorrerlas en paralelo

El optimizador elige según: tamaño de las tablas, presencia de índices, memoria disponible, y si los datos ya están ordenados.

Páginas (repaso)

Las bases de datos no leen filas individuales del disco. Leen **páginas** (bloques de tamaño fijo).

```
Disco:  [Página 0][Página 1][Página 2][Página 3] ...
         8 KB       8 KB       8 KB       8 KB      (en PostgreSQL)
```

- Una página contiene **varias filas** (o fragmentos de columnas).
- Leer 1 fila = leer toda la página donde vive.
- Leer 2 filas en la misma página = **1 sola lectura de I/O**.

> Todo lo que veremos hoy — scans, joins, row stores, column stores — se entiende desde la página.

El rendimiento es sobre I/O de páginas (repaso)

Nivel	Latencia
Cache L1/L2	~1-10 ns
RAM	~100 ns
SSD secuencial	~10 µs
SSD random	~100 µs
HDD	~10 ms

- El costo se mide en **páginas leídas**, no filas
- **I/O secuencial**: páginas consecutivas → el hardware las lee de corrido
- **I/O random**: cada salto paga la latencia completa del medio
- Esto explica todas las decisiones del optimizador que veremos hoy

Operadores de Scan

Operador	Páginas	Cuándo
Sequential Scan	Lee **todas** en orden (I/O secuencial)	Baja selectividad (>5-10% de filas)
Index Scan	Navega índice → salta a páginas con resultados (I/O random)	Alta selectividad (<5%)
Index-Only Scan	Solo lee páginas del índice, no toca la tabla	Solo necesita columnas del índice

¿Por qué no siempre usar el índice? Cada fila puede estar en una página diferente → muchos saltos random. Si necesitamos >5-10% de la tabla, leer todo de corrido es más barato.

Scan: EXPLAIN ANALYZE

Sequential Scan — baja selectividad → lee toda la tabla

```
EXPLAIN ANALYZE SELECT * FROM ventas WHERE monto > 10;
```

```
Seq Scan on ventas  (cost=0.00..21370.00 rows=950000 width=112)
                    (actual time=0.01..95.3 rows=948721 loops=1)
  Filter: (monto > 10)
```

Index Scan — alta selectividad → salta directo al dato

```
EXPLAIN ANALYZE SELECT * FROM ventas WHERE sale_id = 42;
```

```
Index Scan using ventas_pkey on ventas  (cost=0.42..8.44 rows=1 width=112)
                                        (actual time=0.02..0.03 rows=1 loops=1)
  Index Cond: (sale_id = 42)
```

Algoritmos de Join: Nested Loop Join

```
Para cada fila r en R:          ← tabla externa (outer)
    Para cada fila s en S:      ← tabla interna (inner)
        Si r.key == s.key:
            emitir (r, s)
```

- **Sin índice**: por cada fila de R, se leen **todas las páginas** de S → O(pág_R × pág_S) I/Os
- **Con índice en S**: por cada fila de R, se lee ~1 página del índice + 1 página de datos → O(pág_R + |R| × 2) I/Os
- **Cuándo es bueno**: tabla interna pequeña (pocas páginas), o hay un índice en S

Nested Loop: EXPLAIN ANALYZE

```sql
SET enable_hashjoin = off;
SET enable_mergejoin = off;
EXPLAIN ANALYZE
SELECT * FROM ventas v JOIN productos p ON v.product_id = p.product_id
WHERE p.category = 'Electronics';
```

```
Nested Loop  (cost=0.29..45123.50 rows=100000 width=156)
             (actual time=0.05..234.5 rows=98542 loops=1)
  -> Seq Scan on productos p  (rows=100)
       Filter: (category = 'Electronics')
  -> Index Scan using idx_ventas_product on ventas v  (rows=985 loops=100)
       Index Cond: (product_id = p.product_id)
```

El loop externo recorre productos filtrados; el interno usa el índice para buscar ventas.

Algoritmos de Join: Hash Join

```
Fase BUILD:
    Para cada fila s en S (tabla más pequeña):
        insertar s en hash_table[s.key]

Fase PROBE:
    Para cada fila r en R (tabla más grande):
        buscar en hash_table[r.key]
        Si hay match: emitir (r, s)
```

- **I/O**: lee las páginas de cada tabla **exactamente una vez** → O(pág_R + pág_S)
- **Fase BUILD**: carga las páginas de S y construye la hash table en memoria
- **Fase PROBE**: recorre las páginas de R, busca cada fila en la hash table
- **Cuándo es bueno**: no hay índice, la tabla más chica cabe en memoria
- **Si no cabe en RAM**: la hash table se "derrama" a disco (*spill*), generando I/O extra

Hash Join: EXPLAIN ANALYZE

```sql
EXPLAIN ANALYZE
SELECT * FROM ventas v JOIN clientes c ON v.customer_id = c.customer_id;
```

```
Hash Join  (cost=3148.00..28456.00 rows=1000000 width=180)
           (actual time=42.1..312.7 rows=1000000 loops=1)
  Hash Cond: (v.customer_id = c.customer_id)
  -> Seq Scan on ventas v     (rows=1000000)
  -> Hash                     (rows=100000)
       -> Seq Scan on clientes c  (rows=100000)
       Buckets: 131072  Memory Usage: 7021kB
```

PostgreSQL construye la hash table sobre `clientes` (la tabla más chica) y luego recorre `ventas`.

Algoritmos de Join: Sort-Merge Join

```
1. Ordenar R por key
2. Ordenar S por key
3. Recorrer ambas listas ordenadas en paralelo:
      avanzar el puntero que tenga la llave menor
      cuando las claves coinciden: emitir (r, s)
```

- **Ya ordenados**: solo recorrer ambas tablas una vez → O(pág_R + pág_S)
- **NO ordenados**: hay que ordenar primero. Si no cabe en RAM → **external merge sort** (ordenar trozos en memoria, escribir a disco, mezclar) → **muchas páginas extras de I/O**
- **Cuándo es bueno**: datos ya ordenados (por un índice o un ORDER BY previo), o el resultado se necesita ordenado
- Por eso el optimizador evita este join cuando los datos no están pre-ordenados

Sort-Merge Join: EXPLAIN ANALYZE

```sql
SET enable_hashjoin = off;
EXPLAIN ANALYZE
SELECT * FROM ventas v JOIN productos p ON v.product_id = p.product_id;
```

```
Merge Join  (cost=156230.00..178456.00 rows=1000000 width=156)
            (actual time=523.1..892.4 rows=1000000 loops=1)
  Merge Cond: (v.product_id = p.product_id)
  -> Sort  (actual time=412.3..534.2 rows=1000000)
       Sort Key: v.product_id
       Sort Method: external merge  Disk: 112MB
  -> Sort  (actual time=0.8..1.1 rows=1000)
       Sort Key: p.product_id
       Sort Method: quicksort  Memory: 102kB
```

Notemos: `ventas` no cabe en memoria → external merge sort (disco). `productos` sí cabe → quicksort.

Resumen: ¿Cuándo elige cada uno?

Algoritmo	Mejor caso (I/O)	Peor caso (I/O)
Nested Loop	pág_R + |R| × ~2 (con índice)	pág_R × pág_S (sin índice)
Hash Join	pág_R + pág_S (cabe en RAM)	+ spill a disco si no cabe
Sort-Merge	pág_R + pág_S (ya ordenados)	+ costo de sort externo

El optimizador estima costos y elige. **Pero puede equivocarse** si las estadísticas están desactualizadas (e.g., después de un bulk load sin `ANALYZE`).

```
ANALYZE ventas;  -- Siempre actualizar estadísticas después de cargas masivas
```

Leyendo EXPLAIN ANALYZE

```
Hash Join  (cost=A..B rows=C width=D) (actual time=E..F rows=G loops=H)
            ▲           ▲       ▲              ▲          ▲        ▲
       costo estimado   filas   bytes     tiempo real   filas   repeticiones
       (startup..total) estimadas por fila (ms)        reales
```

Campo	Significado
`cost`	Estimación del optimizador (no son ms)
`actual time`	Medición real en ms
`rows`	Estimadas vs reales — si difieren mucho → estadísticas desactualizadas
`Buffers: shared hit/read`	hit = en cache, read = leídas de disco

El plan es un **árbol**: cada nodo consume el output del hijo.

La 2da ejecución puede ser más rápida (buffer cache) — siempre medir en frío.

Parte B

Row Stores vs Column Stores

Del algoritmo al layout de datos

Hemos visto cómo PostgreSQL elige entre algoritmos de join y scan

> **¿Y si el cuello de botella no es el algoritmo, sino cuántos datos leemos del disco?**

Una tabla tiene 20 columnas. Nuestra consulta solo necesita 2.

¿Por qué leer las otras 18?

Almacenamiento orientado a filas (NSM)

Cada **página** contiene filas completas, una tras otra:

```
Página 1:  [fila1: id,nombre,fecha,monto,cat,...] [fila2: id,nombre,fecha,monto,cat,...]
Página 2:  [fila3: id,nombre,fecha,monto,cat,...] [fila4: id,nombre,fecha,monto,cat,...]
     ...
Página N:  [fila999999: ...]  [fila1000000: ...]
```

- `SELECT * WHERE id = 42` → buscar con el índice, leer **1 página** ✅
- `SELECT AVG(monto) FROM ventas` → **1M filas en 20 columnas**, pero necesitamos solo `monto`. Aún así leemos **TODAS las páginas** con todas las columnas ❌

Las cuentas: filas de ~200 bytes, páginas de 8 KB → ~40 filas/página → 1M filas = **25,000 páginas** para un simple `AVG` de 1 columna.

Almacenamiento orientado a columnas (DSM)

Cada **página** contiene valores de **una sola columna**:

```
Páginas de "id":     [1, 2, 3, ..., 2000] [2001, 2002, ...]
Páginas de "nombre": [Ana, Bob, Carlos, ...] [...]
Páginas de "monto":  [150, 230, 89, 445, ...] [...]
Páginas de "cat":    [A, B, A, C, A, B, ...] [...]
```

- `SELECT AVG(monto)` → leer **solo** las páginas de `monto`. Un entero = 4 bytes → ~2,000 valores por página → 1M filas ≈ **500 páginas** (vs 25,000 en row store) ✅
- `SELECT * WHERE id = 42` → encontrar la posición 42 en **cada archivo de columna**, leer una página de cada uno → 20 lecturas random ❌

¿Por qué las columnas ganan para analítica?

1. Menos páginas leídas

Solo leemos las páginas de las columnas que necesitamos.

En una tabla de 20 columnas, leer 2 columnas = **~90% menos páginas de I/O**.

El ejemplo anterior: 500 páginas vs 25,000.

2. Mejor compresión → aún menos páginas

Valores del mismo tipo y dominio juntos → compresión mucho más efectiva.

Una columna `país` con dictionary encoding puede caber en **un puñado de páginas**.

3. Operaciones vectorizadas

Las páginas columnares contienen datos homogéneos → se procesan en bloques:

- Mejor uso de **cache de CPU** (datos contiguos del mismo tipo)
- Potencial para instrucciones **SIMD** (una instrucción, múltiples datos)

Predicate Pushdown y Projection Pushdown

Dos optimizaciones fundamentales — las vamos a encontrar en todo sistema moderno:

Projection pushdown — "Solo lee las columnas que necesito"

```
SELECT monto, fecha FROM ventas WHERE ...
-- Solo lee 2 columnas, ignora las otras 18
```

Predicate pushdown — "Filtra lo antes posible"

```
SELECT * FROM ventas v JOIN productos p ON ...
WHERE v.fecha >= '2025-01-01'
-- El filtro de fecha se aplica ANTES del join, no después
```

Compresión columnar (1/2)

Run-Length Encoding (RLE)

```
[A, A, A, A, B, B, C, C, C]  →  [(A,4), (B,2), (C,3)]
```

Excelente para columnas ordenadas o de baja cardinalidad (ej: `país`, `categoría`).

Dictionary Encoding

```
["Chile","Chile","Argentina","Chile","Argentina"]
→ Diccionario: {0:"Chile", 1:"Argentina"}
→ Datos: [0, 0, 1, 0, 1]
```

Reemplaza strings por enteros. Los warehouses lo aplican automáticamente.

Compresión columnar (2/2)

Delta Encoding

```
[1000, 1001, 1003, 1004, 1010]  →  [1000, +1, +2, +1, +6]
```

Para columnas con valores incrementales (ej: timestamps, IDs secuenciales).

¿Por qué importa?

Mejor compresión = **menos páginas en disco** = menos I/O.

Una columna `país` con dictionary encoding puede reducirse **10-50x** respecto al texto original.

Late Materialization

En un row store:

```
SELECT nombre FROM productos WHERE precio > 100;
```

→ Lee **toda** la fila, filtra, retorna `nombre`.

En un column store:

```
1. Leer columna "precio": [50, 200, 80, 150, 30]
2. Evaluar precio > 100 → posiciones: [1, 3]
3. Leer columna "nombre" SOLO en posiciones [1, 3]
4. Retornar: ["Widget B", "Widget D"]
```

Formato Parquet

El estándar de facto para almacenamiento columnar en archivos.

```
Archivo Parquet
├── Row Group 1 (~128 MB, ~1M filas)
│   ├── Column Chunk: id     [min=1, max=1000000]
│   ├── Column Chunk: monto  [min=0.5, max=9999.9]
│   ├── Column Chunk: fecha  [min=2023-01-01, max=2023-06-30]
│   └── ...
├── Row Group 2
│   └── ...
└── Footer (metadata, schema, estadísticas)
```

- **Row group** (~128 MB): subconjunto de filas, almacenado en columnas
- **Pages** (~1 MB): unidad de lectura y compresión dentro de cada column chunk
- Estadísticas **min/max** por chunk → el motor puede **saltar row groups completos**
- Misma idea que las páginas en una BD, pero a escala de archivos en disco/S3

Parquet: vive en S3/GCS

En la práctica, los archivos Parquet viven en **object storage** (S3, GCS):

```
s3://mi-datalake/ventas/
├── year=2023/month=01/  data_001.parquet
├── year=2023/month=02/  data_002.parquet
├── ...
└── year=2025/month=03/  data_015.parquet
```

Partition pruning: `WHERE year = 2025` → el motor **no abre** los archivos de otros años.

Cada archivo que no abrimos = una request HTTP que nos ahorramos.

Desafíos de Parquet

Tamaño de Row Groups

- Muy chicos → perdemos el beneficio del scan columnar
- Muy grandes → no podemos saltar grupos irrelevantes

Evolución de Schema

- Agregar columnas: OK ✅
- Cambiar tipos: doloroso ❌ (hay que reescribir los archivos)

Problema de archivos pequeños

- Miles de Parquet chicos en S3 = performance terrible
- Cada apertura de archivo = una request HTTP separada
- Solución: **compaction** (unir archivos pequeños en grandes)

¿Cuándo sigue ganando el row store?

Operación	Row store	Column store
Punto: `WHERE id = 42`	✅ 1 página	❌ Lee N columnas
`INSERT` una fila	✅ Append rápido	❌ Escribe en N archivos
`UPDATE` una fila	✅ 1 página	❌ Reescritura costosa
`AVG(x)` sobre toda la tabla	❌ Lee toda la tabla	✅ Solo 1 columna
`GROUP BY a, SUM(b)`	❌ Lee toda la tabla	✅ Solo 2 columnas

Row store → OLTP (transacciones, inserciones, updates puntuales)

Column store → OLAP (análisis, agregaciones, reportes)

Ejecución: Tuple-at-a-Time vs Vectorizada

PostgreSQL — procesa **una fila a la vez**:

```
Leer página → fila 1 → filtro → operador → fila 2 → filtro → operador → ...
```

Cada página mezcla tipos (int, text, date...) → el CPU salta entre formatos.

DuckDB (columnar) — procesa **vectores de ~2048 valores**:

```
Leer páginas de "monto" → [150, 230, 89, 445, ...] (vector completo)
→ evaluar filtro sobre todo el vector de golpe
→ pasar vector al siguiente operador
```

Páginas homogéneas (solo ints, o solo strings) → loop tight, **cache L1/L2**, **SIMD**.

> La diferencia no es solo "columnas vs filas", sino también **cómo el CPU procesa los datos**.

Parte C

DuckDB y el camino a Data Warehousing

DuckDB: "SQLite para analítica"

- Base de datos **columnar**, **in-process** (sin servidor)
- Lee Parquet nativamente — sin "cargar" datos primero
- No tiene índices B-tree tradicionales — usa **zone maps** (min/max por row group) y late materialization
- Motor de ejecución vectorizado
- Ideal para análisis en un laptop

```
import duckdb
# Consulta directa sobre un archivo Parquet:
duckdb.sql("SELECT AVG(monto) FROM 'ventas.parquet' WHERE fecha >= '2025-01-01'")
```

La **misma consulta**, dos motores diferentes:

```sql
SELECT p.category, SUM(s.amount), COUNT(*)
FROM sales s JOIN products p ON s.product_id = p.product_id
WHERE s.sale_date >= '2025-01-01'
GROUP BY p.category
ORDER BY SUM(s.amount) DESC;
```

	PostgreSQL	DuckDB
Storage	Row-oriented	Column-oriented
Execution	Tuple-at-a-time	Vectorized
Plan	Hash Join + Seq Scan	Hash Join + Column Scan
Indexes	B-tree, Hash	Zone maps

Las diferencias vienen de **storage layout + modelo de ejecución**, no solo "columnas son más rápidas".

El camino a Data Warehousing

Acabamos de ver que los **column stores** son dramáticamente mejores para consultas analíticas.

Ahora imaginemos:

- Nuestro sistema transaccional (PostgreSQL, row-store) maneja órdenes todo el día
- Nuestro analista quiere: `SELECT region, SUM(revenue) GROUP BY region` sobre **3 años de datos**
- ¿Deberíamos ejecutar eso en la base de producción? **No.**

→ Necesitamos un **sistema separado**, optimizado para estas consultas analíticas.
→ **Eso es un Data Warehouse.**

Warehousing

Todo lo que vimos hoy es la base de los warehouses modernos:

Concepto de hoy	En Data Warehousing
Almacenamiento columnar	BigQuery, Snowflake, Redshift — todos columnares
Zone maps (min/max)	Reemplazan índices tradicionales a escala
Parquet en S3	Separación storage/compute
Compresión columnar	RLE, dictionary encoding automáticos
Predicate/projection pushdown	Las optimizaciones clave en motores distribuidos